

Bases de Dados

Introdução ao SQL



Sumário

1

Revisão do Modelo Lógico

2

Introdução à SQL

3

Descrição de Dados – DDL

4

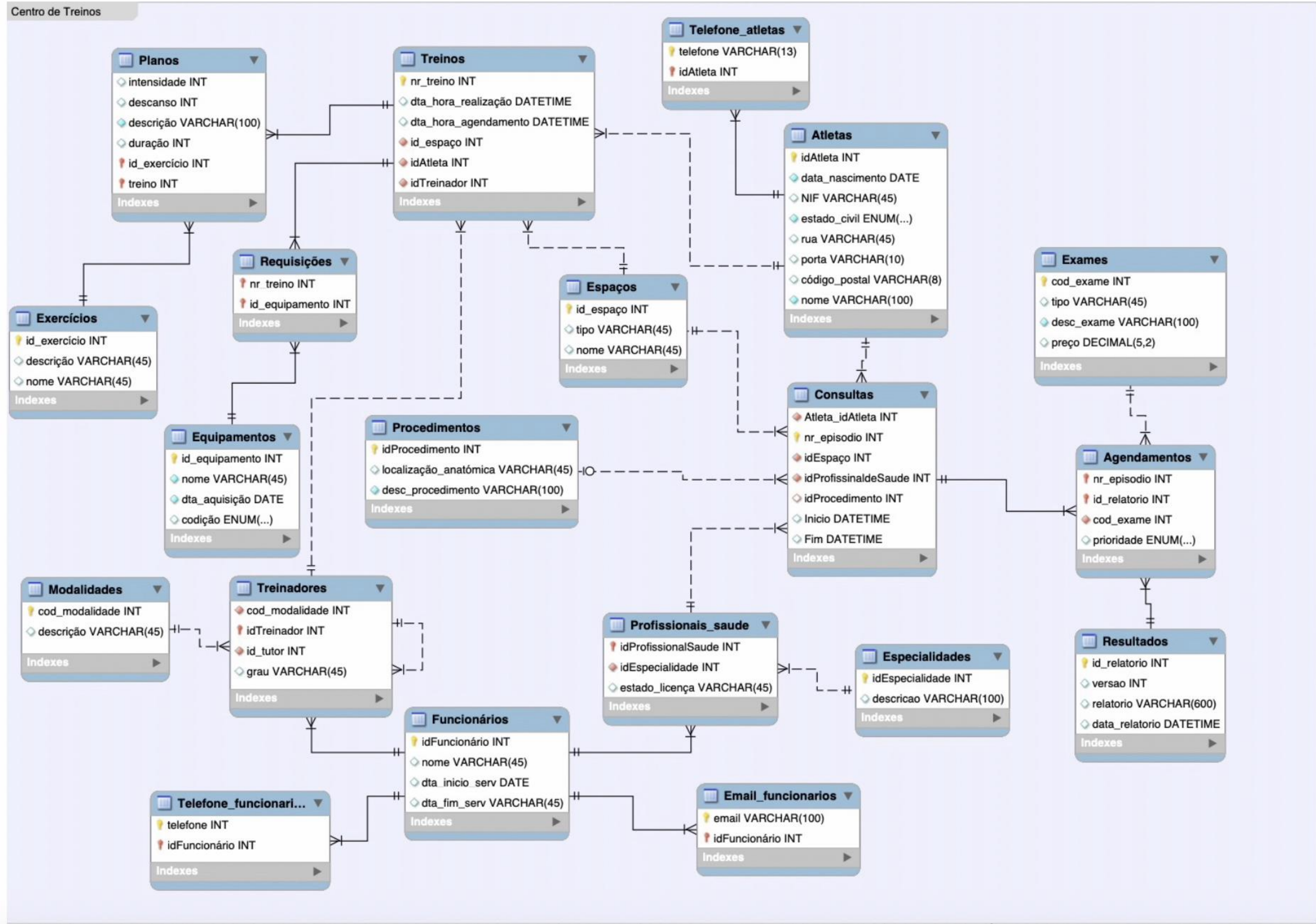
Atualização e Manipulação de
Dados em SQL – DML

Bibliografia:

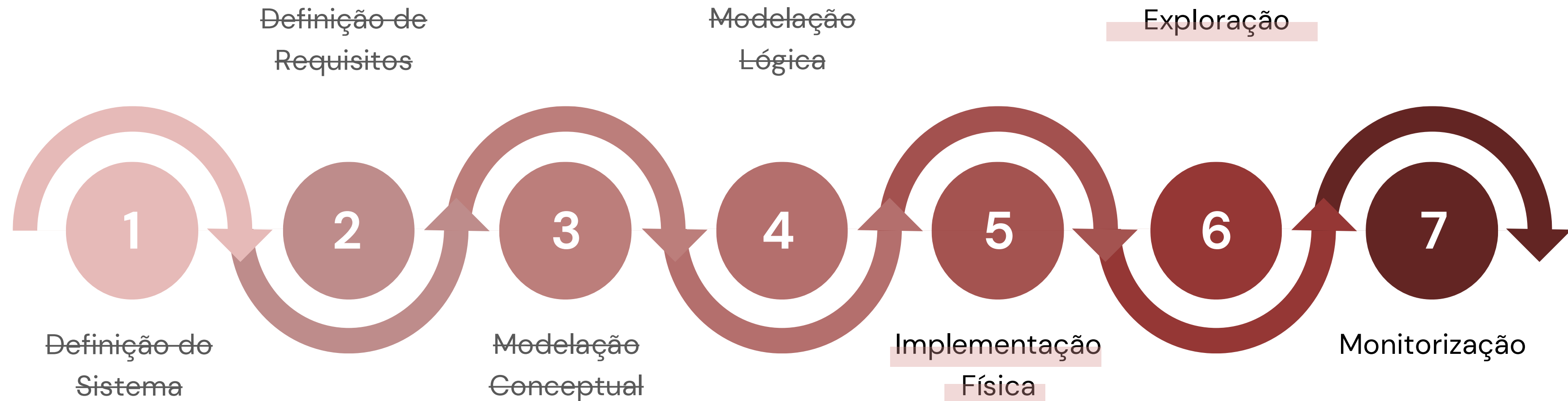
- Connolly, T., Begg, C., Database Systems, A Practical Approach to Design, Implementation, and Management , Addison-Wesley, 4a Edição, 2004. **(Chapter 18)**
- Belo, O., “Bases de Dados Relacionais: Implementação com MySQL”, FCA – Editora de Informática, 376p, Set 2021. ISBN: 978-972-722-921-5. **(Chapter 2)**

Modelo Relacional

Caso de Estudo



Ciclo de vida de um SBD



Linguagens de Bases de Dados

➔ Introdução

As bases de dados necessitam de mecanismos que permitam aos utilizadores e aplicações interagir com os dados de forma estruturada e eficiente.

Esses mecanismos são fornecidos através de **linguagens de bases de dados**, que devem permitir a um utilizador:

- criar a base de dados e as estruturas das relações;
- executar tarefas de gestão de dados básicos, tais como a inserção, alteração e exclusão de dados das relações;
- realizar consultas simples e complexas.

Uma linguagem de bases de dados deve executar essas tarefas com esforço mínimo do utilizador, e a sua estrutura e sintaxe devem ser fáceis de aprender.

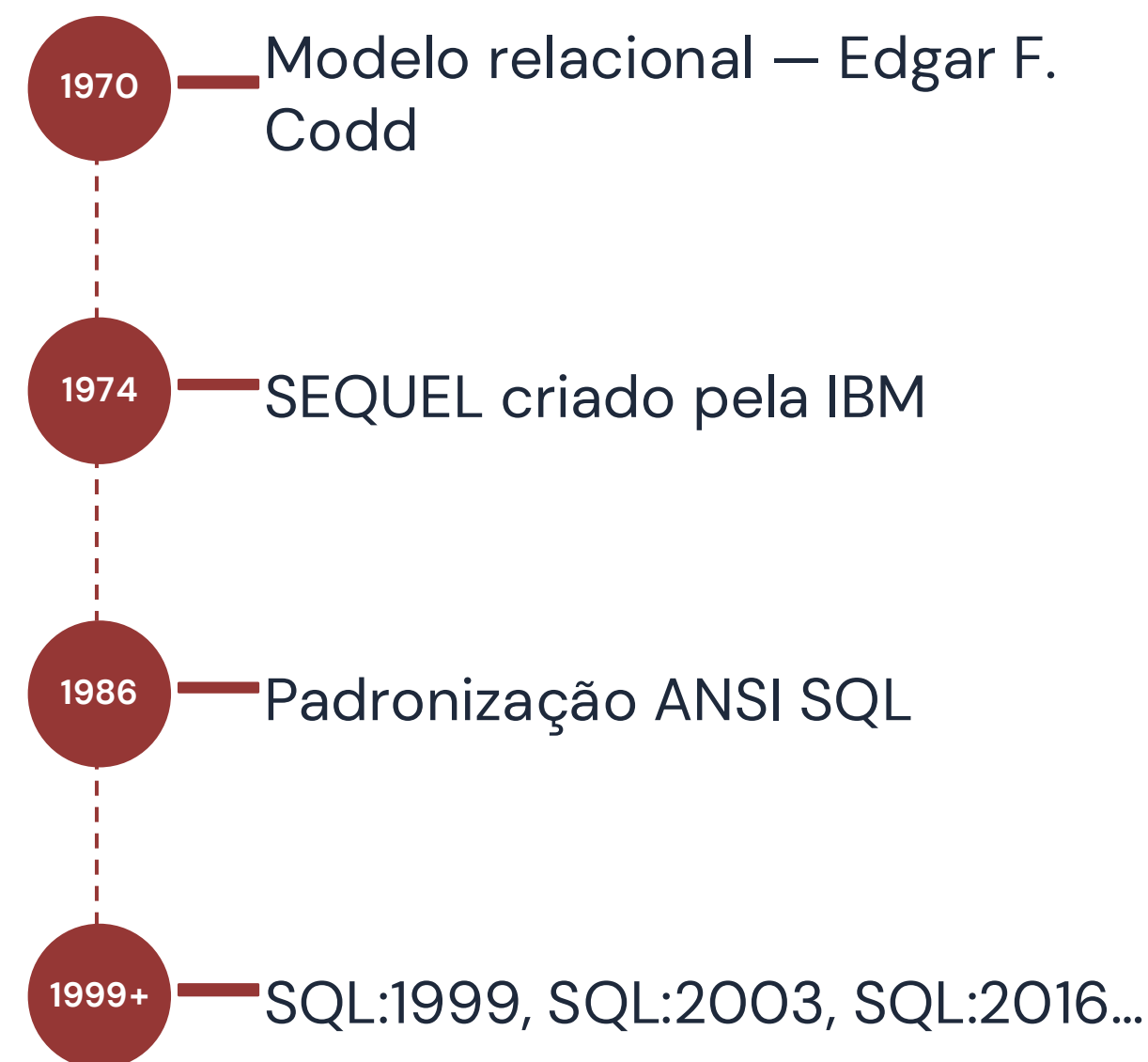
A Linguagem SQL

➔ O que é SQL?

SQL (Structured Query Language) é a linguagem padrão para interagir com Sistemas de Gestão de Bases de Dados Relacionais (SGBDR).

Foi desenvolvida pela IBM nos anos 70 e padronizada pelo ANSI/ISO.

É declarativa: especifica **O QUE** pretendemos obter, não **COMO** obter.



A Linguagem SQL

➔ SGBDs

O SQL é suportado pela maioria dos Sistemas de Gestão de Bases de Dados (SGBD), tais como:



Apesar de existirem pequenas variações entre sistemas (dialetos), a base da linguagem mantém-se comum.

O SQL é amplamente utilizado em:

- Aplicações web
- Sistemas empresariais
- Aplicações móveis
- Sistemas de análise de dados

A Linguagem SQL

➔ SGBDs

O MySQL é o SGBDR que será utilizado ao longo desta unidade curricular.

Com o MySQL é possível:

- Criar e estruturar bases de dados
- Armazenar e gerir informação
- Executar consultas SQL
- Garantir integridade e segurança dos dados



O MySQL é uma ferramenta open-source, amplamente utilizada tanto em contexto académico como na indústria.

É atualmente mantido pela Oracle Corporation.

A Linguagem SQL

➔ Operações

DDL Data Definition Language

Define a estrutura da base de dados

CREATE
ALTER
DROP

DML Data Manipulation Language

Manipula os dados

INSERT
UPDATE
DELETE

DQL Data Query Language

Consulta os dados:

SELECT

TCL Data Control Language

Controla permissões de acesso

GRANT
REVOKE

A Linguagem SQL

DDL

➔ Descrição de Dados

A DDL (Data Definition Language) é utilizada para **definir e modificar a estrutura da base de dados**.

Permite criar, alterar e eliminar objetos da base de dados, tais como:

- Tabelas
- Índices
- Esquemas
- Vistas

Os comandos DDL atuam diretamente sobre a **estrutura** e não sobre os dados em si.
Principais comandos:

- CREATE
- ALTER
- DROP
- TRUNCATE

A Linguagem SQL

DDL

➔ Criar Base de Dados

CREATE DATABASE

O comando `CREATE DATABASE` é utilizado para criar uma nova base de dados.

Estrutura

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
[create_option] ...
create_option: [DEFAULT] {
CHARACTER SET [=] charset_name
| COLLATE [=] collation_name
| ENCRYPTION [=] {'Y' | 'N'}}
```

A Linguagem SQL

DDL

➔ Criar Base de Dados

CREATE DATABASE

O comando `CREATE DATABASE` é utilizado para criar uma nova base de dados.

Exemplo:

```
CREATE DATABASE IF NOT EXISTS escola;
```

Esta parte é opcional

Neste caso é criada uma base de dados com o nome escola

Para utilizar a base de dados criada:

```
USE escola;
```

A partir deste momento, todas as operações serão realizadas dentro desta base de dados.

`DROP DATABASE` – remoção de uma base de dados de um sistema.

A Linguagem SQL

DDL

➔ Criar Tabelas

Tabelas

Nos sistemas de bases de dados relacionais, a informação é armazenada em tabelas (também designadas por relações).

As principais operações de definição de dados sobre tabelas são:

- **CREATE TABLE** — permite criar a estrutura de uma nova tabela na base de dados
- **ALTER TABLE** — permite modificar a estrutura de uma tabela existente
- **DROP TABLE** — permite remover uma tabela da base de dados

A Linguagem SQL

DDL

➔ Criar Tabelas

CREATE TABLE

```
CREATE TABLE [IF NOT EXISTS ] <nome_tabela> (  
  <nome_coluna> <tipo_coluna[tamanho]> [NOT NULL | NULL] [DEFAULT <value>]  
  [AUTO_INCREMENT][UNIQUE],  
  ...,  
  PRIMARY KEY (<nome_coluna_PK>,...)  
  [CONSTRAINT <constraint_name>] UNIQUE {KEY | INDEX} (<nome_coluna>,...)  
  [CONSTRAINT <constraint_name>] FOREIGN KEY (<nome_coluna_FK>) REFERENCES <nome_tabela_FK>  
  (<nome_coluna_FK>)  
  [ON UPDATE <referential_integrity_constraint>] [ON DELETE  
  <referential_integrity_constraint>]  
) [ENGINE=<storage_engine>];
```

Se o ENGINE não for declarado, o MySQL usará o InnoDB por padrão.

referential_integrity_constraint = {NO ACTION | RESTRICT | CASCADE | SET NULL | SET DEFAULT } Se não especificar a cláusula ON DELETE e ON UPDATE, o MySQL usará a definição padrão.

A Linguagem SQL

DDL

➔ Descrição de Dados

Tipos de dados

Em SQL, cada coluna de uma tabela deve ter um **tipo de dados**, que define o tipo de valores que pode armazenar (domínio). Os principais tipos de dados são:

Numéricos

INT — números inteiros

DECIMAL(p,s) — números com casas decimais (precisão fixa)

FLOAT — números reais (aproximados)

Texto

CHAR(n) — texto de tamanho fixo

VARCHAR(n) — texto de tamanho variável

Booleano

BOOLEAN — valores verdadeiro/falso (em MySQL: 0 ou 1)

Data e Hora

DATE — data (YYYY-MM-DD)

TIME — hora

DATETIME — data e hora

Outros

BLOB — dados binários (ex: imagens, ficheiros)

TEXT — textos longos

A Linguagem SQL

DDL

➔ Descrição de Dados

Restrições

As restrições (constraints) são regras aplicadas às colunas de uma tabela para garantir a integridade e consistência dos dados. Permitem controlar que tipo de valores podem ser inseridos na base de dados.

PRIMARY KEY

- Identifica unicamente cada registo
- Não pode conter valores NULL
- Não pode ter valores duplicados

NOT NULL

- Garante que a coluna não pode ter valores vazios

UNIQUE

- Impede valores duplicados numa coluna

UNSIGNED

- Impede valores negativos

FOREIGN KEY

- Estabelece ligação entre tabelas
- Garante integridade referencial

CHECK

- Define uma condição que os valores devem respeitar

DEFAULT

- Define um valor por defeito para a coluna

A Linguagem SQL

DDL

➔ Criar Tabelas

CREATE TABLE

O comando CREATE TABLE permite criar uma nova tabela dentro da base de dados.

```
CREATE TABLE Aluno (  
  id      INT      AUTO_INCREMENT PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL,  
  email   VARCHAR(150) UNIQUE,  
  data_nasc DATE  
);
```

Exemplo 1:

- É criada a tabela alunos
- São definidos os atributos e tipos de dados
- A coluna id é definida como chave primária
- O valor de id é gerado automaticamente a cada novo registo
- O nome é obrigatório
- E o email não pode repetir

A Linguagem SQL

DDL

➔ Criar Tabelas

CREATE TABLE

```
CREATE TABLE Curso (  
  id      INT      PRIMARY KEY,  
  descrição VARCHAR(150) NOT NULL,  
  limite  INT      CHECK (limite >= 0),  
  estado  VARCHAR(20) DEFAULT 'ativo'  
);
```

Exemplo 2:

- É criada a tabela curso
- São definidos os atributos e tipos de dados
- A coluna id é definida como chave primária
- A limite tem uma condição
- O estado tem um valor *default*

A Linguagem SQL

DDL

➔ Criar Tabelas

CREATE TABLE

```
CREATE TABLE Inscrições (  
  aluno_id    INT,  
  curso_id    INT,  
  PRIMARY KEY (aluno_id, curso_id),  
  FOREIGN KEY (aluno_id) REFERENCES alunos(id),  
  FOREIGN KEY (curso_id) REFERENCES cursos(id)  
);
```

Exemplo 3:

- É criada a tabela inscrições
- São definidos os atributos e tipos de dados
- A chave primária é composta
- São definidas 2 chaves estrangeiras (Integridade referencial)

A Linguagem SQL

DDL

➔ Alteração de Tabelas

Após a criação de uma tabela, pode surgir a necessidade de **modificar a sua estrutura**. Essas alterações podem ocorrer por vários motivos:

- Novos requisitos do sistema
- Correção de erros de modelação
- Evolução da base de dados ao longo do tempo

Para realizar estas modificações, o SQL disponibiliza o comando:

```
ALTER TABLE
```

Este comando permite:

- Adicionar novas colunas
- Remover colunas existentes
- Alterar tipos de dados
- Adicionar ou remover restrições

A Linguagem SQL

DDL

➔ Alteração de Tabelas

ALTER TABLE

ALTER TABLE <nome_tabela_antigo> RENAME TO <nome_tabela_novo>; → altera o nome da tabela.

ALTER TABLE <nome_tabela> ADD <nome_campo> <domínio_campo>; → cria um novo atributo na tabela com o nome e domínio especificados. Todos os tuplos existentes ficam com NULL no novo atributo.

ALTER TABLE <nome_tabela> DROP <nome_campo>; → apaga o atributo com o nome especificado da tabela.

ALTER TABLE <nome_tabela> MODIFY <nome_campo> <domínio_campo>; → modifica o atributo com o nome especificado da tabela.

ALTER TABLE <nome_tabela> ALTER < nome_campo> SET DEFAULT <value>; → modifica uma coluna de uma tabela para lhe atribuir valores padrão.

ALTER TABLE <nome_tabela> ALTER < nome_campo> DROP DEFAULT; → remove os valores padrão de uma coluna de uma tabela.

ALTER TABLE <nome_tabela> ADD CONSTRAINT <nome_constraint> UNIQUE KEY(column_1,column_2,...); → modifica uma tabela para lhe atribuir uma indexação única .

A Linguagem SQL

DDL

➔ Alteração de Tabelas

ALTER TABLE

Exemplos:

- Adição de um novo atributo telefone à tabela alunos:

```
ALTER TABLE alunos  
ADD NIF INT;
```

- Adição de novos atributos com indicação de posição:

```
ALTER TABLE alunos  
ADD telefone VARCHAR(13) AFTER nome,  
ADD data_nascimento DATE FIRST;
```

- Remoção de um atributo de uma tabela:

```
ALTER TABLE alunos  
DROP COLUMN idade;
```

- Modificação da definição de um atributo:

```
ALTER TABLE alunos  
MODIFY nome VARCHAR(150) NOT NULL;
```

- Modificação de vários atributos de uma tabela:

```
ALTER TABLE alunos  
MODIFY nome VARCHAR(150) NOT NULL,  
MODIFY idade INT UNSIGNED;
```

- Alteração do nome de um atributo:

```
ALTER TABLE alunos  
CHANGE nome nome_completo VARCHAR(150);
```


A Linguagem SQL

DDL

➔ Remoção de Tabelas

A remoção de uma tabela de uma base de dados é realizada através da instrução:

```
DROP TABLE
```

Quando uma tabela é removida, são eliminados:

- Todos os registos (dados)
- Índices e gatilhos (triggers)
- Restrições (constraints)
- Privilégios associados
- Toda a sua definição na base de dados

⚠ **Importante:**

Se uma tabela estiver associada a uma **chave estrangeira**, não poderá ser removida diretamente.

Nesse caso, é necessário:

- Remover previamente a restrição de chave estrangeira
- Ou eliminar as dependências existentes

A Linguagem SQL

DDL

➔ Remoção de Tabelas

DROP TABLE

Exemplos:

- Remoção de uma tabela:

```
DROP TABLE alunos;
```

- Remoção de várias tabelas:

```
DROP TABLE alunos, cursos;
```

- Remoção apenas se a tabela existir:

```
DROP TABLE IF EXISTS alunos;
```

A Linguagem SQL

DML

➔ Manipulação de Dados

A DML (Data Manipulation Language) corresponde ao conjunto de instruções SQL utilizadas para **manipular os dados armazenados nas tabelas**.

Estas instruções permitem:

- Inserir novos registos
- Modificar dados existentes
- Remover dados da base de dados

As operações DML atuam diretamente sobre o **conteúdo das tabelas**, não alterando a sua estrutura.

Principais instruções:

- INSERT — inserção de dados
- UPDATE — modificação de dados
- DELETE — remoção de dados

A Linguagem SQL

DML

➔ Inserção de Dados

A inserção de dados numa tabela é realizada através da instrução:

```
INSERT INTO
```

Estrutura:

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name
[PARTITION (partition_name [, partition_name] ...)]
[(col_name [, col_name] ...)]
{ {VALUES | VALUE} (value_list) [, (value_list)] ... }
[AS row_alias[(col_alias [, col_alias] ...)]]
[ON DUPLICATE KEY UPDATE assignment_list]
(...)
```

A Linguagem SQL

DML

➔ Inserção de Dados

INSERT INTO

Exemplos:

- Inserção de um novo registo:

```
INSERT INTO Curso (id, descrição, limite, estado)
VALUES (1, 'Programação', 30, 'ativo');
```

OU

```
INSERT INTO Curso VALUES (1, 'Programação', 30, 'ativo');
```

- Inserção de vários registos:

```
INSERT INTO Curso (id, descrição, limite)
VALUES
(2, 'Biologia', 20),
(3, 'Música', 10);
```

Neste caso o atributo estado não é especificado → fica com o valor 'ativo' pois tinha sido especificado como *default*

A Linguagem SQL

DML

➔ Inserção de Dados

INSERT INTO

Exemplos:

Inserção de um registo numa tabela, especificando apenas alguns dos atributos definidos no esquema da tabela:

```
INSERT INTO Aluno (nome, email)
VALUES ('Ana Silva', 'ana@email.com');
```

Neste caso:

O atributo id é gerado automaticamente (AUTO_INCREMENT)

O atributo data_nasc não é especificado → fica com valor NULL

A Linguagem SQL

DML

➔ Modificação de Dados

A modificação de dados é realizada através da instrução:

UPDATE

Estrutura:

```
UPDATE [LOW_PRIORITY] [IGNORE] table_reference
SET assignment_list
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
value:
{expr | DEFAULT}
assignment:
col_name = value
assignment_list:
assignment [, assignment] ...
```

A Linguagem SQL

DML

➔ Modificação de Dados

UPDATE

Exemplos

- Atualização de um registo (afeta todas as linhas da tabela):

```
UPDATE cursos  
SET estado = 'ativo',
```

- Atualização de vários atributos com uma condição (afeta apenas as linhas que cumprem a condição):

```
UPDATE cursos  
SET estado = 'inativo', limite = '0'  
WHERE id IN (1,2);
```



Sem a cláusula `WHERE`, todos os registos da tabela serão modificados.

A Linguagem SQL

DML

➔ Remoção de Dados

A remoção de dados é realizada através da instrução:

DELETE

Estrutura:

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name [[AS] tbl_alias]
[PARTITION (partition_name [, partition_name] ...)]
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
```

A Linguagem SQL

DML

➔ Remoção de Dados

DELETE

Exemplos

- Remoção de um registo:

```
DELETE FROM alunos  
WHERE id = 1;
```

- Remoção de vários registos:

```
DELETE FROM alunos  
WHERE YEAR(data_nasc) < 2018;
```

- Remoção de todos os registos de uma tabela:

```
DELETE FROM alunos;
```

```
-- Desativar modo de segurança  
SET SQL_SAFE_UPDATES=0;  
-- Ativar modo de segurança  
SET SQL_SAFE_UPDATES=1;
```

⚠ Sem a cláusula WHERE, todos os registos da tabela serão removidos.

Bases de Dados

Introdução ao SQL

